Black Box Testing Report

QuickCart API — Part 3 Complete Test Suite

Roll Number: 2024101087

`Execution baseline: pytest -q blackbox/tests -rxX`

3. Black Box Testing — All 94 Test Cases

Test Case ID	Test Name	API / Endpoint	Input	Expected Output	Justification for Inclusion
BB-01	test_bb01	/cart/add	POST /cart/add product_id=27 qty=2, then product_id=27 qty=3	200 OK; single cart item with quantity=5	Validates cart merge logic — adding same product twice must accumulate quantity not duplicate
BB-02	test_bb02	/cart/add	POST /cart/add quantity=0; then quantity=-1	400 for both zero and negative quantity	Validates lower bound on cart quantity — zero/negative stock additions must be rejected
BB-03	test_bb03	/cart/add	POST /cart/add product_id=999999 quantity=1	404 with error='Product not found'	Validates product existence check before cart insertion
BB-04	test_bb04	/cart/add	POST /cart/add body={quantity:1} (no product_id)	400 validation error for missing required field	Validates required field enforcement on cart add endpoint
BB-05	test_bb05	/cart/update	POST /cart/update product_id=27 quantity=0	400 error payload	Validates zero quantity rejection on cart update — mirrors add validation
BB-06	test_bb06	/cart/remove	POST /cart/remove product_id=27 (not in cart)	404 error mentioning Product and cart	Validates not-found handling on remove for product absent from cart
BB-07	test_bb07	/cart/clear	DELETE /cart/clear after adding product_id=27	200; items=[], total=0	Validates full cart wipe — cart must be completely empty after clear
BB-08	test_bb08	/cart	Add product_id=1 qty=2 and product_id=3 qty=5; GET /cart	subtotal[1]=240, subtotal[3]=200, total=440	Validates billing arithmetic — subtotals and total must follow unit_price×quantity
BB-09	test_bb09	/coupon/apply	Add product_id=1 qty=1; POST /coupon/apply coupon_code=WELCOME50	200; coupon_code=WELCOME50, discount=50, new_total=70	Validates fixed-amount coupon application and discounted total calculation
BB-10	test_bb10	/coupon/apply + /checkout	Apply WELCOME50, then POST /checkout payment_method=CARD	200; total_amount = (120-50) + 5% GST = 73.5	Validates coupon discount is carried through to checkout final total
BB-11	test_bb11	/coupon/apply	Add product_id=1 qty=1; apply BONUS75 (min threshold not met)	400 with 'below minimum' in error	Validates minimum cart value enforcement for coupon eligibility
BB-12	test_bb12	/coupon/apply	Add product_id=1 qty=10; apply SUPER10	200; discount=80, new_total=1120	Validates max-cap enforcement for percentage coupons
BB-13	test_bb13	/coupon/apply	Add product_id=1 qty=10; apply EXPIRED100	400 with 'expired' in error	Validates coupon expiry date check — expired codes must never be accepted
BB-14	test_bb14	/checkout	POST /checkout with empty cart, payment_method=CARD	400 with 'Cart is empty' in error	Validates checkout precondition — non-empty cart must be enforced
BB-15	test_bb15	/checkout	Add item; POST /checkout payment_method=UPI	400 error='Invalid payment method'	Validates payment method enum — only CARD/COD/WALLET accepted
BB-16	test_bb16	/checkout	Add product_id=1 qty=1; POST /checkout payment_method=CARD	200; gst_amount=6, total_amount=126, payment_status=PAID, order_status=PLACED	Validates CARD checkout: GST computation and payment status mapping
BB-17	test_bb17	/checkout	Add product_id=1 qty=50; POST /checkout payment_method=COD	400 with 'COD not allowed' in error	Validates COD business rule — high-value orders must be rejected for cash on delivery
BB-18	test_bb18	/checkout	Add product_id=1 qty=1; POST /checkout payment_method=WALLET	200; gst_amount=6, total_amount=126, payment_status=PENDING	Validates WALLET checkout: payment status mapping and GST application
BB-19	test_bb19	/admin/users	GET /admin/users without X-Roll-Number	401 with 'X-Roll-Number' in error	Validates mandatory global authentication header for all admin routes
BB-20	test_bb20	/admin/users	GET /admin/users X-Roll-Number: abc	400 with 'valid integer' in error	Validates strict roll-number integer parsing
BB-21	test_bb21	/profile	GET /profile without X-User-ID header	400 with 'X-User-ID' in error	Confirms user-scoped endpoints require user identity header
BB-22	test_bb22	/profile	GET /profile X-User-ID: not-a-number	400 with 'valid positive integer' in error	Covers non-numeric user ID input validation
BB-23	test_bb23	/profile	GET /profile X-User-ID: 999999	404 error='User not found'	Validates not-found response for unrecognised user IDs
BB-24	test_bb24	/admin/users/{id}	GET /admin/users then GET /admin/users/{id} for first listed user	200; detail response == listing entry	Verifies consistency between list and detail admin endpoints
BB-25	test_bb25	/admin/users + /profile	GET /admin/users/{id} and GET /profile for same user_id	200; all fields match between admin and public profile views	Cross-validates admin and user-facing data consistency
BB-26	test_bb26	/admin/users	GET /admin/users	200; non-empty list of users	Basic smoke test confirming admin users endpoint is seeded and operational
BB-27	test_bb27	/admin/products	GET /admin/products	200; product_id=90 present in list	Confirms admin view includes inactive products hidden from public storefront
BB-28	test_bb28	/profile	GET /profile for clean user	200; keys: user_id, name, email, phone, wallet_balance, loyalty_points	Validates response schema — all documented fields must be present
BB-29	test_bb29	/addresses	GET /addresses for user with addresses	200; list instance	Basic smoke test confirming addresses endpoint returns correct data type
BB-30	test_bb30	/profile	PUT /profile body={name:'Only Name'}	400 error	Validates that phone is a required field in profile update
BB-31	test_bb31	/profile	PUT /profile body={}	400 error	Validates empty payload rejection for profile update
BB-32	test_bb32	/profile	PUT /profile name=12345, phone='1234567890'	400 error	Validates type enforcement — name must be a string not numeric
BB-33	test_bb33	/profile	PUT /profile name='Valid Name', phone=1234567890 (integer)	400 error	Validates type enforcement — phone must be string not integer
BB-34	test_bb34	/addresses	POST /addresses label='BAD'	400 error	Validates enum constraint — only valid label values accepted
BB-35	test_bb35	/addresses	POST /addresses street='1234' (4 chars)	400 error	Validates minimum street length boundary
BB-36	test_bb36	/addresses	POST /addresses street=101-char string	400 error	Validates maximum street length boundary
BB-37	test_bb37	/addresses	POST /addresses city='D' (1 char)	400 error	Validates minimum city length boundary
BB-38	test_bb38	/addresses	POST /addresses city=51-char string	400 error	Validates maximum city length boundary

Test Case ID	Test Name	API / Endpoint	Input	Expected Output	Justification for Inclusion
BB-39	test_bb39	/addresses	POST /addresses pincode='12345' (5 digits)	400 error	Validates pincode minimum length — must be exactly 6 digits
BB-40	test_bb40	/addresses	POST /addresses pincode='1234567' (7 digits)	400 error	Validates pincode maximum length — must be exactly 6 digits
BB-41	test_bb41	/addresses	POST /addresses body without label key	400 error	Validates that label is a required field
BB-42	test_bb42	/addresses	POST /addresses body without street key	400 error	Validates that street is a required field
BB-43	test_bb43	/addresses	POST /addresses body without city key	400 error	Validates that city is a required field
BB-44	test_bb44	/addresses	POST /addresses body without pincode key	400 error	Validates that pincode is a required field
BB-45	test_bb45	/addresses	POST /addresses label=123 (integer)	400 error	Validates type enforcement — label must be string
BB-46	test_bb46	/addresses	POST /addresses street=12345 (integer)	400 error	Validates type enforcement — street must be string
BB-47	test_bb47	/addresses	POST /addresses pincode=123456 (integer not string)	400 error	Validates type enforcement — pincode must be string
BB-48	test_bb48	/addresses	POST /addresses is_default='true' (string not boolean)	400 error	Validates type enforcement — is_default must be boolean
BB-49	test_bb49	/cart/add	POST /cart/add product_id='1' (string), quantity=1	400 error	Validates numeric type enforcement for product_id field
BB-50	test_bb50	/cart/add	POST /cart/add product_id=1, quantity='1' (string)	400 error	Validates numeric type enforcement for quantity field
BB-51	test_bb51	/checkout	POST /checkout body={}	400 error	Validates that payment_method is a required checkout field
BB-52	test_bb52	/checkout	POST /checkout payment_method=null	400 error	Validates null input rejection for required payment_method field
BB-53	test_bb53	/wallet/add	POST /wallet/add amount=100001	400 error	Validates upper bound for wallet top-up amount
BB-54	test_bb54	/wallet/add	POST /wallet/add body={}	400 error	Validates required amount field enforcement for wallet top-up
BB-55	test_bb55	/products/{id}/reviews	POST /products/250/reviews rating=6, comment='too high'	400 error	Validates rating upper bound — max allowed is 5
BB-56	test_bb56	/products/{id}/reviews	POST /products/250/reviews rating='5' (string)	400 error	Validates type enforcement — rating must be integer not string
BB-57	test_bb57	/products/{id}/reviews	POST /products/250/reviews rating=4, comment=201-char string	400 error	Validates comment maximum length boundary enforcement
BB-58	test_bb58	/products/{id}/reviews	POST /products/250/reviews rating=4 (no comment key)	400 error	Validates that comment is a required review field
BB-59	test_bb59	/support/tickets/{id}	Create ticket; PUT /support/tickets/{id} status='REOPENED'	400 error	Validates strict status enum — REOPENED is not a valid support ticket status
BB-60	test_bb60	/support/tickets/{id}	PUT /support/tickets/999999 status=IN_PROGRESS	404 error	Validates missing-resource semantics for ticket update
BB-61	test_bb61	/profile	PUT /profile name='QA User', phone='1234567890'	200; GET /profile returns updated name and phone	Validates happy-path profile update and persistence
BB-62	test_bb62	/profile	PUT /profile name='Q', phone='1234567890'	400; error='Name must be between 2 and 50 characters'	Validates lower bound on name — minimum 2 characters required
BB-63	test_bb63	/profile	PUT /profile name='Valid User', phone='12345' (5 digits)	400; error='Phone must be exactly 10 digits'	Validates exact phone length enforcement
BB-64	test_bb64	/profile	PUT /profile name='Valid Name', phone='98A654321B' (non-digit chars)	400; error='Phone must be exactly 10 digits'	Validates phone digit-only constraint beyond length check
BB-65	test_bb65	/addresses	POST /addresses label=OFFICE, street, city, pincode=123456, is_default=false	200; message='Address added successfully'; full address object returned	Validates happy-path address creation and complete response schema
BB-66	test_bb66	/addresses	POST /addresses pincode='500001'	200; address.pincode='500001'	Validates valid pincode acceptance — runtime can occasionally reject valid codes
BB-67	test_bb67	/addresses	POST /addresses pincode='ABCDE'	400; 'Pincode' in error	Validates that pincode must contain only digits
BB-68	test_bb68	/addresses/{id}	Create address; PUT /addresses/{id} street='54321 Updated Avenue', is_default=false	200; street updated; label/city/pincode unchanged from original	Validates partial update — only sent fields change, others remain intact
BB-69	test_bb69	/addresses/{id}	DELETE /addresses/999999	404; error='Address not found'	Validates missing-resource semantics for delete operation
BB-70	test_bb70	/addresses	POST /addresses is_default=true for second address	200; only new address has is_default=true; exactly 1 default total	Validates business rule — only one address can be the default at any time
BB-71	test_bb71	/products	GET /products	200; all items have is_active=true; product_id=90 (inactive) absent	Validates storefront visibility rule — inactive products must be hidden
BB-72	test_bb72	/products/{id}	GET /products/999999	404; error='Product not found'	Validates missing-product handling for single product lookup
BB-73	test_bb73	/products	GET /products?search=Apple; GET /products?category=Vegetables	200; search returns Apple products; category returns only Vegetables	Validates storefront discovery — search and category filters must work correctly
BB-74	test_bb74	/products + /admin/products	GET /products; GET /admin/products; compare prices by product_id	All public prices exactly match admin source prices	Detects price transformation defects between admin data and public API
BB-75	test_bb75	/products	GET /products?sort=asc; GET /products?sort=desc	asc: prices non-decreasing; desc: prices non-increasing	Validates product sort order — storefront must respect sort parameter
BB-76	test_bb76	/products/{id}/reviews	GET /products/250/reviews (no reviews seeded)	200; reviews=[], average_rating=0	Validates no-data aggregate behavior — zero average when review list is empty
BB-77	test_bb77	/products/{id}/reviews	POST reviews rating=4 and rating=5; GET /products/250/reviews	200; average_rating=4.5; 2 reviews in list	Validates aggregate decimal precision — average must not be truncated
BB-78	test_bb78	/products/{id}/reviews	POST /products/250/reviews rating=0, comment='bad'	400; 'between 1 and 5' in error	Validates rating lower bound — minimum valid rating is 1

Test Case ID	Test Name	API / Endpoint	Input	Expected Output	Justification for Inclusion
BB-79	test_bb79	/products/{id}/reviews	POST /products/250/reviews rating=4, comment=''	400; 'between 1 and 200 characters' in error	Validates non-empty comment constraint — empty string must be rejected
BB-80	test_bb80	/support/ticket	POST /support/ticket subject='Refund help', message='Need a refund because...'	200; body.message and admin ticket message match exactly	Validates that support messages are stored without truncation
BB-81	test_bb81	/support/tickets	Create ticket → IN_PROGRESS → CLOSED → OPEN (backward)	IN_PROGRESS: 200; CLOSED: 200; OPEN after CLOSED: 400 'Invalid status transition'	Validates support ticket state-machine — backward transitions must be blocked
BB-82	test_bb82	/support/tickets	Create ticket; PUT status=CLOSED immediately	400; 'Invalid status transition' in error	Validates that tickets cannot skip IN_PROGRESS state
BB-83	test_bb83	/support/ticket	POST /support/ticket subject='Shrt' (4 chars), message='x'	400; 'between 5 and 100 characters' in error	Validates subject minimum length boundary for ticket creation
BB-84	test_bb84	/wallet	GET /wallet (initial); POST /wallet/add amount=50	200; new wallet_balance = initial + 50	Validates basic wallet top-up and balance update
BB-85	test_bb85	/wallet/add	POST /wallet/add amount=0; POST /wallet/add amount=100001	400 for both	Validates lower and upper bounds for wallet top-up amount
BB-86	test_bb86	/wallet/pay	POST /wallet/pay amount=100 (balance insufficient)	400; 'Insufficient' in error	Validates wallet overdraft protection — cannot pay more than balance
BB-87	test_bb87	/wallet/pay	GET /wallet for user_id=1; POST /wallet/pay amount=balance+1	400; 'Insufficient' in error	Validates per-user balance enforcement for wallet debit
BB-88	test_bb88	/wallet/pay	GET initial balance; POST /wallet/add 50; POST /wallet/pay 25	200; wallet_balance = initial + 25 (exact deduction)	Validates precise debit accounting — no hidden fees or rounding errors
BB-89	test_bb89	/loyalty/redeem	GET /loyalty (initial points); POST /loyalty/redeem points=10	200; loyalty_points = initial - 10	Validates loyalty points deduction on valid redemption
BB-90	test_bb90	/loyalty/redeem	POST /loyalty/redeem points=0; POST /loyalty/redeem points=current+1	400 for both	Validates minimum (zero) and insufficient-points rejection boundaries
BB-91	test_bb91	/orders + /checkout	Checkout product_id=1 qty=1; GET /orders; GET /orders/{id}; GET /orders/{id}/invoice	order in list; detail total=126; invoice subtotal=120, gst=6, total=126	Validates order creation consistency across list, detail, and invoice endpoints
BB-92	test_bb92	/orders/{id}/cancel + /products	Add products 1&3; checkout; POST /orders/{id}/cancel	200 status=CANCELLED; stock restored for both products	Validates inventory compensation on cancellation
BB-93	test_bb93	/orders/{id}/cancel	POST /orders/999999/cancel	404; error='Order not found'	Validates missing-resource handling for order cancellation
BB-94	test_bb94	/orders/{id}/cancel	POST /orders/{delivered_order_id}/cancel	400; error='Cannot cancel delivered order'	Validates terminal order state — delivered orders must not be cancellable

4. Bug Report

The following 23 bug reports correspond to the 16 XFAIL (strict=True) confirmed defects, 4 XFAIL (strict=False) flaky defects, and 3 XPASS results observed during the test run.

4.1 Confirmed Bugs — XFAIL strict=True (16 defects)

BR-01: Cart accepts zero and negative add-to-cart quantities [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/cart/add**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/cart/add
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "product_id": 1, "quantity": 0 }
- **Expected result:**
  - HTTP 400 — Invalid quantity error payload
- **Actual result observed:**
  - HTTP 200 — Zero and negative quantities accepted silently
  - Assertion failure: assert zero_quantity.status_code == 400 → received 200

BR-02: Cart add missing product_id returns 404 instead of 400 validation error [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/cart/add**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/cart/add
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "quantity": 1 }
- **Expected result:**
  - HTTP 400 — validation error for missing product_id
- **Actual result observed:**
  - HTTP 404 — Product not found (product_id treated as absent/0)
  - Assertion failure: assert response.status_code == 400 → received 404

BR-03: Cart subtotals overflow and total drops items [XFAIL (strict=True)]

- **Endpoint tested: GET /api/v1/cart (after POST /api/v1/cart/add)**
- **Request payload:**
  - Method: GET
  - URL: http://localhost:8080/api/v1/cart
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: none
- **Expected result:**
  - subtotal[product_id=1]=240, subtotal[product_id=3]=200, total=440
- **Actual result observed:**
  - Subtotals overflow to negative values; total drops item contributions
  - Assertion failure: assert item_by_id[1]['subtotal'] == 240 → received -16

BR-04: Checkout total ignores applied coupon discount [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/coupon/apply + POST /api/v1/checkout**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/checkout
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "payment_method": "CARD" } (after applying WELCOME50)

- **Expected result:**
  - total_amount = (120 - 50) + 5% GST = 73.5
- **Actual result observed:**
  - Checkout ignores coupon; charges full price + GST = 126.0
  - Assertion failure: assert checkout_response.json()['total_amount'] == 73.5 → received 126.0

BR-05: Expired coupon EXPIRED100 accepted when cart value qualifies [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/coupon/apply**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/coupon/apply
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "coupon_code": "EXPIRED100" }
- **Expected result:**
  - HTTP 400 — Coupon expired error
- **Actual result observed:**
  - HTTP 200 — Coupon accepted despite being expired
  - Assertion failure: assert response.status_code == 400 → received 200

BR-06: Checkout proceeds with empty cart instead of rejecting [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/checkout**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/checkout
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "payment_method": "CARD" }
- **Expected result:**
  - HTTP 400 — 'Cart is empty' error
- **Actual result observed:**
  - HTTP 200 — Order placed even with empty cart
  - Assertion failure: assert response.status_code == 400 → received 200

BR-07: Profile update accepts non-digit phone values (10-char but not all digits) [XFAIL (strict=True)]

- **Endpoint tested: PUT /api/v1/profile**
- **Request payload:**
  - Method: PUT
  - URL: http://localhost:8080/api/v1/profile
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "name": "Valid Name", "phone": "98A654321B" }
- **Expected result:**
  - HTTP 400 — 'Phone must be exactly 10 digits'
- **Actual result observed:**
  - HTTP 200 — Non-digit characters accepted when total length is 10
  - Assertion failure: assert response.status_code == 400 → received 200

BR-08: Multiple default addresses allowed after creating new default [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/addresses**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/addresses
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "label": "OTHER", "street": "...", "city": "Pune", "pincode": "654321", "is_default": true }
- **Expected result:**
  - HTTP 200; only 1 address has is_default=true after creation
- **Actual result observed:**

  - Multiple addresses remain marked as default; previous default not cleared
  - Assertion failure: assert len(default_addresses) == 1 → received > 1

BR-09: Public product prices do not match admin/seeded prices [XFAIL (strict=True)]

- **Endpoint tested: GET /api/v1/products (vs GET /api/v1/admin/products)**
- **Request payload:**
  - Method: GET
  - URL: http://localhost:8080/api/v1/products
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: none
- **Expected result:**
  - All public prices exactly match admin seeded prices
- **Actual result observed:**
  - Several product prices are rounded away from admin values (e.g. 225.05 → 230)
  - Assertion failure: assert product['price'] == admin_products[id]['price'] → 230 != 225.05

BR-10: Product list ignores requested price sort order [XFAIL (strict=True)]

- **Endpoint tested: GET /api/v1/products?sort=asc / ?sort=desc**
- **Request payload:**
  - Method: GET
  - URL: http://localhost:8080/api/v1/products?sort=asc
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: none
- **Expected result:**
  - asc: ascending prices; desc: descending prices
- **Actual result observed:**
  - Sort parameter ignored; products returned in insertion/default order
  - Assertion failure: assert ascending_prices == sorted(ascending_prices) → order mismatch

BR-11: Review rating=6 (above maximum) accepted instead of rejected [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/products/250/reviews**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/products/250/reviews
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 29`
  - Body: { "rating": 6, "comment": "too high" }
- **Expected result:**
  - HTTP 400 — rating must be between 1 and 5
- **Actual result observed:**
  - HTTP 200 — Rating 6 accepted
  - Assertion failure: assert response.status_code == 400 → received 200

BR-12: Review rating=0 (below minimum) accepted instead of rejected [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/products/250/reviews**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/products/250/reviews
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 29`
  - Body: { "rating": 0, "comment": "bad" }
- **Expected result:**
  - HTTP 400 — 'between 1 and 5' in error
- **Actual result observed:**
  - HTTP 200 — Rating 0 accepted
  - Assertion failure: assert response.status_code == 400 → received 200

BR-13: Support ticket message truncated instead of saved exactly [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/support/ticket**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/support/ticket
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 29`
  - Body: { "subject": "Refund help", "message": "Need a refund because the delivered apples were bruised." }
- **Expected result:**
  - body.message and admin ticket message match input exactly
- **Actual result observed:**
  - Message saved truncated; admin ticket shows shorter version
  - Assertion failure: assert saved_ticket['message'] == message → strings differ (truncated)

BR-14: Wallet pay over-deducts instead of charging exact amount [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/wallet/pay**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/wallet/pay
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 29`
  - Body: { "amount": 25 } (after adding 50)
- **Expected result:**
  - wallet_balance = initial + 25 (exact debit of 25)
- **Actual result observed:**
  - wallet_balance lower than expected — hidden extra deduction applied
  - Assertion failure: assert response.json()['wallet_balance'] == approx(initial + 25) → mismatch

BR-15: Invoice total does not match checkout total for fresh orders [XFAIL (strict=True)]

- **Endpoint tested: GET /api/v1/orders/{id}/invoice**
- **Request payload:**
  - Method: GET
  - URL: http://localhost:8080/api/v1/orders/{order_id}/invoice
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 31`
  - Body: none
- **Expected result:**
  - invoice: subtotal=120, gst_amount=6, total_amount=126
- **Actual result observed:**
  - Invoice fields do not match checkout response for the same order
  - Assertion failure: assert invoice['total_amount'] == 126 → mismatch

BR-16: Server hangs when cancelling a newly placed order [XFAIL (strict=True)]

- **Endpoint tested: POST /api/v1/orders/{id}/cancel**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/orders/{order_id}/cancel
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 33`
  - Body: none
- **Expected result:**
  - HTTP 200; order_status=CANCELLED; stock restored for all products
- **Actual result observed:**
  - Server hangs on cancel request — timeout exceeded (10s); stock not restored
  - Assertion failure: response.status_code == 200 → timeout / no response

4.2 Flaky Bugs — XFAIL strict=False (4 defects)

BR-17: Valid 6-digit pincode 500001 occasionally rejected (flaky) [XFAIL (strict=False)]

- **Endpoint tested: POST /api/v1/addresses**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/addresses
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "label": "OTHER", "street": "Main Street 12", "city": "Hyderabad", "pincode": "500001", "is_default": false }
- **Expected result:**
  - HTTP 200; address.pincode='500001'
- **Actual result observed:**
  - Intermittently returns HTTP 400 'Invalid pincode' for valid 6-digit codes
  - Assertion failure: assert response.status_code == 200 → occasionally 400

BR-18: Non-digit pincode values occasionally accepted instead of rejected (flaky) [XFAIL (strict=False)]

- **Endpoint tested: POST /api/v1/addresses**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/addresses
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "label": "HOME", "street": "Main Street 12", "city": "Hyderabad", "pincode": "ABCDE", "is_default": false }
- **Expected result:**
  - HTTP 400 — 'Pincode' in error
- **Actual result observed:**
  - Intermittently returns HTTP 200; non-digit pincode accepted
  - Assertion failure: assert response.status_code == 400 → occasionally 200

BR-19: Wallet pay above current balance intermittently succeeds (flaky) [XFAIL (strict=False)]

- **Endpoint tested: POST /api/v1/wallet/pay**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/wallet/pay
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 1`
  - Body: { "amount": current_balance + 1 }
- **Expected result:**
  - HTTP 400 — 'Insufficient' in error
- **Actual result observed:**
  - Intermittently returns HTTP 200; payment above balance succeeds
  - Assertion failure: assert response.status_code == 400 → occasionally 200

BR-20: Support ticket allows direct OPEN→CLOSED transition (flaky) [XFAIL (strict=False)]

- **Endpoint tested: PUT /api/v1/support/tickets/{id}**
- **Request payload:**
  - Method: PUT
  - URL: http://localhost:8080/api/v1/support/tickets/{ticket_id}
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 29`
  - Body: { "status": "CLOSED" } (ticket still OPEN)
- **Expected result:**
  - HTTP 400 — 'Invalid status transition'
- **Actual result observed:**
  - Intermittently returns HTTP 200; skips IN_PROGRESS state
  - Assertion failure: assert closed.status_code == 400 → occasionally 200

4.3 Unexpected Passes — XPASS (3 results)

BR-21: Valid pincode 500001 accepted correctly (XPASS — bug did not reproduce) [XPASS]

- **Endpoint tested: POST /api/v1/addresses**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/addresses
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "label": "OTHER", "street": "Main Street 12", "city": "Hyderabad", "pincode": "500001", "is_default": false }
- **Expected result:**
  - HTTP 200 (was expected to fail as per flaky bug report)
- **Actual result observed:**
  - HTTP 200 — valid pincode accepted; bug did not trigger in this run
  - Assertion failure: Test passed unexpectedly (XPASS) — intermittent bug not reproduced

BR-22: Non-digit pincode correctly rejected (XPASS — bug did not reproduce) [XPASS]

- **Endpoint tested: POST /api/v1/addresses**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/addresses
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 7`
  - Body: { "label": "HOME", "pincode": "ABCDE", ... }
- **Expected result:**
  - HTTP 400 (was expected to sometimes succeed as per flaky report)
- **Actual result observed:**
  - HTTP 400 — non-digit pincode correctly rejected; bug did not reproduce
  - Assertion failure: Test passed unexpectedly (XPASS) — intermittent acceptance not reproduced

BR-23: Wallet pay above balance correctly rejected (XPASS — bug did not reproduce) [XPASS]

- **Endpoint tested: POST /api/v1/wallet/pay**
- **Request payload:**
  - Method: POST
  - URL: http://localhost:8080/api/v1/wallet/pay
  - Headers:
    - `X-Roll-Number: 2024101087`
    - `X-User-ID: 1`
  - Body: { "amount": current_balance + 1 }
- **Expected result:**
  - HTTP 400 (was expected to sometimes succeed as per flaky report)
- **Actual result observed:**
  - HTTP 400 — overdraft correctly rejected; bug did not reproduce in this run
  - Assertion failure: Test passed unexpectedly (XPASS) — intermittent success not reproduced